UNIT-5 CHAPTER-2
BIG DATA

What is Big Data?

- **Big Data refers to extremely large and complex datasets that cannot be processed using traditional database systems.**
- **These datasets are so massive that they require specialized tools and technologies for storage, processing, and analysis.**

Key Characteristics of Big Data (The 3Vs Model)

1. **Volume → Refers to the huge amount of data generated every second (e.g., social media posts, transactions, sensor data).**
2. **Velocity → Data is generated at high speed and needs to be processed quickly (e.g., stock market transactions, real-time analytics).**
3. **Variety → Data comes in different formats (structured, semi-structured, and unstructured).**

Sometimes, two more Vs are added:

4. **Veracity → Ensures the accuracy and reliability of data.**
5. **Value → Extracting useful insights from Big Data.**

Examples of Big Data

- **Social media platforms like Facebook, Twitter, and Instagram generate petabytes of data daily.**
- **E-commerce websites (Amazon, Flipkart) track millions of customer transactions and preferences.**
- **Healthcare systems generate massive medical records, patient histories, and genetic data.**

TYPES OF BIG DATA

Big Data is categorized into three types based on its structure:
1 Structured Data
2 Semi-Structured Data
3 Unstructured Data

Structured Data

- **Structured Data is well-organized and stored in fixed formats like tables and databases.**
- **It follows a specific schema (rows and columns), making it easy to store, search, and analyze.**

Examples of Structured Data:

- **Relational Databases (MySQL, PostgreSQL)**
- **Spreadsheets (Excel, Google Sheets)**
- **Bank Transactions (Account numbers, balances, timestamps, amounts, etc.)**
- **Employee Records (Name, Age, Salary, Department, etc.)**

Advantages of Structured Data:
✅ **Easy to search and retrieve → Since it is stored in tables, we can use SQL queries to access data quickly.**
✅ **Highly organized → Data is stored in a logical format, making it easier to manage.**
✅ **Efficient for analysis → Works well with data mining and business intelligence tools.**

Disadvantages of Structured Data:
❌ **Limited flexibility → Cannot handle complex or changing data formats.**
❌ **Difficult to scale → Traditional databases struggle with huge volumes of data.**

Semi-Structured Data

- **Semi-Structured Data has some structure but does not follow a strict table format like structured data.**
- **It contains tags, markers, or metadata that help organize the data.**

Examples of Semi-Structured Data:

- **JSON (JavaScript Object Notation) → Used in APIs, web services**
- **XML (Extensible Markup Language) → Used in web data storage**
- **Emails → Contains structured headers (To, From, Subject) and unstructured body text**
- **CSV Files → Data stored in comma-separated values, but not in a fixed relational database**
- **Log Files → Server and application logs contain timestamped entries but are not fully structured**

Advantages of Semi-Structured Data:
✅ **More flexible than structured data → Can store varied and evolving data formats.**
✅ **Easier to analyze than unstructured data → Metadata helps in processing and searching data.**
✅ **Can be stored in NoSQL databases like MongoDB and Cassandra.**

Disadvantages of Semi-Structured Data:
❌ **Not as easily searchable as structured data → Queries are more complex.**
❌ **Requires extra processing → Needs specialized tools to organize and analyze the data.**

Unstructured Data

- **Unstructured Data has no predefined format or structure.**
- **It is the largest type of Big Data, accounting for more than 80% of all data generated today.**

Examples of Unstructured Data:

- **Text Documents (Word files, PDFs, notes)**
- **Images and Videos (Instagram posts, CCTV footage)**
- **Social Media Posts (Tweets, Facebook posts, YouTube comments)**
- **IoT Sensor Data (Smart devices, GPS location data)**
- **Medical Imaging (X-rays, MRI scans)**

Advantages of Unstructured Data:

✅ **Huge source of insights → Contains rich information useful for machine learning and AI.**
✅ **Can store complex and multimedia data → Images, videos, and audio files.**
✅ **Scales easily → Cloud-based systems like AWS, Google Cloud can store petabytes of unstructured data.**

Disadvantages of Unstructured Data:

❌ **Difficult to analyze → Cannot be processed using traditional database methods.**
❌ **Storage issues → Requires large amounts of storage space and special tools like Hadoop and Spark.**
❌ **Time-consuming processing → Requires AI and machine learning to extract useful patterns.**

Comparison of Structured, Semi-Structured, and Unstructured Data

Feature	Structured Data	Semi-Structured Data	Unstructured Data
Definition	Organized in **tables (rows & columns)**	Partially organized with **tags, markers, or metadata**	No fixed format or organization
Storage	**Relational Databases (SQL)**	**NoSQL Databases (MongoDB, Cassandra)**	**Cloud Storage, Data Lakes (Hadoop, AWS S3)**
Examples	Banking transactions, Employee records	JSON, XML, Email, Log Files	Social media posts, Videos, Images, IoT data
Ease of Analysis	**Easy** (can be queried with SQL)	**Moderate** (requires special tools)	**Difficult** (requires AI & ML)
Flexibility	**Low** (fixed schema)	**Moderate** (some structure)	**High** (can store anything)
Processing Tools	MySQL, PostgreSQL	MongoDB, Cassandra	Hadoop, Spark, AI algorithms

---

DISTRIBUTED ARCHITECTURE:

HADOOP

Why Do We Need Hadoop? (Motivation for Hadoop)

1. Traditional Data Storage is Expensive

- Storing **large volumes of data** using traditional storage systems is **too costly**.
- Costs include:
  ✅ **High price per GB** for storage.
  ✅ **Expensive hardware** to process massive datasets.
  ✅ **Computational costs** associated with traditional data processing.

Example:

- A company collecting **millions of user interactions** daily needs **a cost-effective way** to store and analyze data.

2. Traditional ETL and Data Warehousing Lose Valuable Data

- The **traditional approach** involves:
  1 **Extracting** data from multiple sources.
  2 **Transforming** and summarizing it.
  3 **Loading it into a Data Warehouse** for analysis.
- ❌ **Problem:**
  - Summarizing and storing only processed data **removes raw details**.
  - **Valuable insights may be lost** in the summarization process.

Example:

- Summarizing **10,000 social media posts** into a single data point **loses critical trends** in user behavior.
- **More data means deeper understanding** – small details matter in Big Data analytics.

3. We Need a Scalable and Reliable System

- **Challenges with Traditional Tools:**
  ✅ **Too much data** to store efficiently.
  ✅ **Difficult to process large amounts of data in real-time.**
  ✅ **High costs make data storage impractical.**
- **Key Questions:**
  ❓ How can we store massive amounts of data **cheaply** and **reliably**?
  ❓ How can we analyze all the data we store?

Solution: Hadoop – A Framework for Distributed Data Storage and Processing

Hadoop Overview:

Definition of Hadoop (By Different Organizations)

1 Google:
"Hadoop is an open-source software framework written in Java for distributed storage and processing of very large datasets on computer clusters built from commodity hardware."
2 Cloudera:
"A free, open-source software framework that supports data-intensive distributed applications."
3 TechTarget:
"Hadoop is a free, Java-based programming framework that supports the processing of large datasets in a distributed computing environment."

Key Features of Hadoop

✅ **Open Source** → Freely available for anyone to use.
✅ **Handles Large Data Volumes** → Designed for Big Data applications.
✅ **Distributed Computing** → Processes data across multiple servers simultaneously.
✅ **Fault Tolerant** → If a node (server) fails, Hadoop automatically recovers.
✅ **Scalable** → More servers (nodes) can be added easily to handle increasing data loads.

Hadoop Components (Core Parts of Hadoop)

Hadoop consists of two core components:

1 Hadoop Distributed File System (HDFS) – Storage Layer

- A distributed file system that spreads data across multiple nodes in a cluster.
- Designed to store large files efficiently.
- Ensures fault tolerance by replicating data across different servers.

2 MapReduce – Processing Layer

- A programming model that processes data in parallel across multiple servers.
- Splits large tasks into smaller tasks and distributes them across the cluster.
- Ensures fast and reliable data processing.

Other Supporting Components:

- Job Scheduling & Monitoring → Manages tasks in a Hadoop cluster.
- Filesystem Utilities → Helps in managing data in HDFS.

Scalability in Hadoop:

- Hadoop uses clusters of servers to store and process data in parallel. It is highly scalable.
- Adding more servers (nodes) increases capacity and performance.

How Hadoop Handles Scalability?

✅ **Distributed System** → Hadoop runs on multiple machines (nodes) instead of a single expensive server.
✅ **Cluster-Based Architecture** → A group of machines running Hadoop together is called a cluster.
✅ **Nodes Work Together** → Each node in a cluster stores data and processes tasks.
✅ **Linear Scalability** → If a cluster has 10 nodes and you add 10 more nodes, the storage and processing power doubles.

Examples of Hadoop Scalability in Action

- Facebook & Yahoo run Hadoop clusters with over 4,400 nodes, proving that Hadoop scales for massive workloads.

Bottom Line: Hadoop can scale from a single machine to thousands of machines effortlessly!

Reliability and Availability in Hadoop:

Why is Reliability Important?

- Traditional database systems require expensive redundancy setups to ensure availability.
- Hadoop provides built-in reliability at a lower cost.

How Hadoop Ensures Reliability & Availability?

✅ **Data is Replicated** → If a server fails, another copy of the data exists elsewhere.
✅ **Tasks are Automatically Rescheduled** → If a task fails, Hadoop assigns it to another node.
✅ **Minimal Human Intervention** → Hadoop automatically detects and handles failures.

Example:

- At Google, servers are attached to racks using Velcro instead of screws, making replacement fast and easy.
- If a server fails, Hadoop redirects work to another node, preventing downtime.

Bottom Line: Hadoop makes data storage and processing highly reliable and available.

Fault Tolerance in Hadoop:

What is Fault Tolerance?

- Fault tolerance means a system continues functioning even if some parts fail.
- In Hadoop, failures are expected but handled automatically.

How Hadoop Ensures Fault Tolerance?

✅ **HDFS Replicates Data Across Multiple Nodes**

- Each file is broken into blocks (usually 64MB each).
- Each block is replicated (default: 3 copies) across different nodes.
- If one node fails, another copy is available elsewhere.

✅ **Tasks are Split into Smaller Parts**

- Hadoop breaks processing jobs into smaller tasks.
- If a node fails during processing, its tasks are reassigned to another node.

Example:

- If a single node in a 100-node cluster fails, Hadoop immediately starts using data from other replicas, ensuring zero data loss.

Bottom Line: Hadoop automatically recovers from failures without human intervention.

How Hadoop Provides Scalability, Reliability, and Fault Tolerance?

Feature	How Hadoop Achieves It?
Scalability	Hadoop clusters grow by adding **more nodes**, scaling **storage and processing power**.
Reliability	**Data is replicated**, so if one node fails, another copy exists.
Fault Tolerance	**Tasks and data are redistributed** automatically in case of failure.

Consequences of this architecture:

1. If a machine fails (even an entire rack) no data is lost
2. Tasks can run elsewhere, where the data resides
3. If a task fails, it can be dispatched to one of the nodes containing redundant copies of the same data block
4. When a failed node comes back online, it can automatically be reintegrated with the cluster

---

HADOOP DISTRIBUTED FILE SYSTEM (HDFS)

What is HDFS?

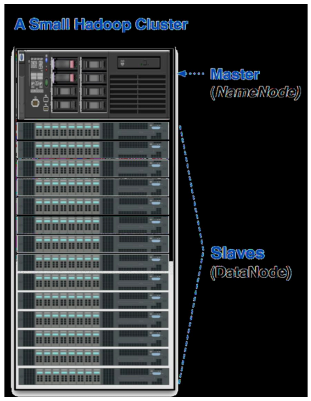
- **HDFS (Hadoop Distributed File System) is** the storage system used by Hadoop to store and manage large datasets efficiently.

- It is designed to provide **inexpensive and reliable storage** for **massive amounts of data**.
- **HDFS is optimized for large files**, typically **100MB or more**, and it **stores files across multiple servers (nodes) in a Hadoop cluster**.
- Cannot modify files once written

Key Features of HDFS

✅ **Stores Large Files Efficiently** → Can handle **multi-gigabyte or terabyte-sized files**.
✅ **Hierarchical Directory Structure** → Files are stored **similar to a traditional file system**.
✅ **Write-Once, Read-Many Model** → Once a file is stored in HDFS, it **cannot be modified**. If changes are needed, the file **must be deleted and recreated**.
✅ **Fault-Tolerant** → If a node fails, **data is automatically recovered** from other nodes.
✅ **Uses Commodity Hardware** → Runs on **low-cost servers** instead of expensive, high-end machines.

HDFS Architecture: Master-Slave Model:



HDFS follows a **Master-Slave architecture** consisting of **two key components**:

1. NameNode (Master Daemon)

- **Stores metadata** → Keeps track of **file names, block locations, and storage locations**.
- **Manages namespace** → Maps **files to blocks** and **blocks to nodes**.
- **Monitors DataNodes** → Ensures **data replication and consistency**.

Example:

- If a file is **300MB**, and HDFS is configured for **128MB blocks**, then:
  - The file will be **split into 3 blocks** (`128MB + 128MB + 44MB`).
  - The **NameNode stores the metadata** of these blocks.

2. DataNode (Slave Daemon)

- **Stores actual file blocks** and **handles read/write requests** from clients.
- Sends **block reports** to NameNode periodically.
- **Replicates data** across multiple nodes to ensure **fault tolerance**.

Example:

- If a file is **stored as 3 blocks**, each block **is copied 3 times** across different DataNodes.
- If **one DataNode fails**, the file **is still accessible from the other copies**.

HDFS Architecture (Detailed Breakdown)

Example: File Storage in HDFS

- Consider two files:
    - **foo.txt (300MB)**
    - **bar.txt (200MB)**
- **HDFS Configuration → 128MB block size**
- The files will be **split into blocks** and distributed across **multiple DataNodes**:

- The files will be **split into blocks** and distributed across **multiple DataNodes**:

File Name	File Size	Number of Blocks (128MB each)
foo.txt	300MB	3 blocks (128MB + 128MB + 44MB)
bar.txt	200MB	2 blocks (128MB + 72MB)

How HDFS Ensures Data Availability?

✅ **Each block is replicated across 3 nodes** by default.
✅ **Block reports** are sent to NameNode periodically to ensure all blocks are available.
✅ **If a DataNode fails**, the **missing block is copied from another DataNode**.

Bottom Line: HDFS ensures high availability by replicating data across multiple nodes.

Why HDFS Prefers Large Files Over Small Files?

HDFS is **optimized for storing large files**, not millions of small files.

Why?

- The **NameNode stores metadata for each file and block**.
- **Too many files or too many small blocks** can **overload the NameNode's memory**.
- **Processing each block requires a Java Virtual Machine (JVM)**, so **too many blocks can slow down performance**.

Best Practice: Store fewer, larger files instead of billions of small files.

Example:

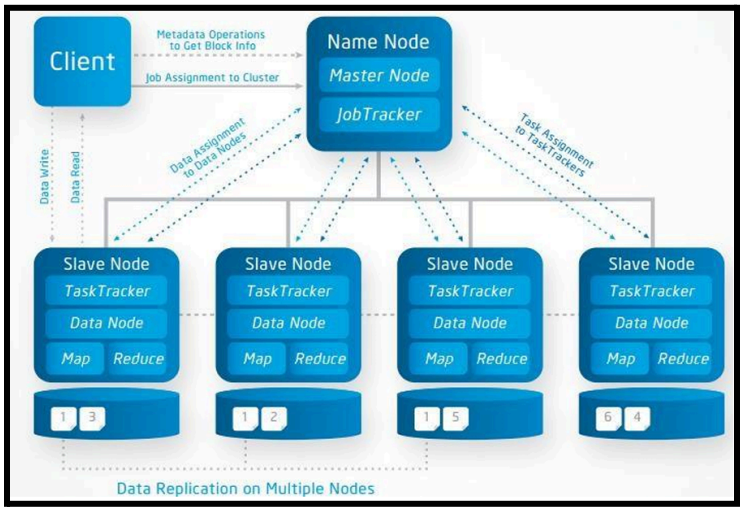
- Storing **10 million 1KB files** will consume too much metadata space in the NameNode.
- Instead, **combining files into larger blocks** improves efficiency.

HDFS Read & Write Operations

Key Components in Hadoop Cluster:

- **NameNode, Secondary NameNode, and Job Tracker → Master Daemons**.
- **Task Tracker and DataNodes → Slave Daemons**.

How Read & Write Works in HDFS?



Write Operation in HDFS

1. **Client sends a write request** to NameNode.
2. **NameNode assigns DataNodes** to store the file.
3. **Data is split into blocks** and **replicated across multiple DataNodes**.
4. **Once all blocks are written, the file is successfully stored**.

Example:

- A **600MB file is written to HDFS**.
- It is split into **5 blocks (128MB each + remaining data)**.
- The blocks are stored across **multiple DataNodes**, ensuring fault tolerance.

Read Operation in HDFS

1. **Client requests a file from the NameNode**.
2. **NameNode checks metadata** and finds which DataNodes store the file blocks.
3. **Client retrieves blocks from DataNodes** and reassembles the file.

Example:

- A **user wants to read a 600MB file** stored in HDFS.
- The **NameNode finds the blocks** and tells the client where they are stored.
- The **client fetches the blocks from multiple DataNodes and reconstructs the file**.

Bottom Line: HDFS efficiently reads and writes large files using a distributed approach.

HDFS Fault Tolerance

Why is Fault Tolerance Important?

- In large-scale clusters, **failures are common** (hardware crashes, network issues, etc.).
- HDFS **ensures data is never lost, even if nodes fail**.

How HDFS Handles Faults?

✅ **Replication of Data Blocks** → Each file block is stored on **3 different nodes by default**.
✅ **If a node fails, HDFS copies the lost data from another node**.
✅ **If a processing task fails, it is automatically reassigned to another node**.
✅ **Heartbeats & Block Reports** → NameNode continuously monitors DataNodes.
✅ **Secondary NameNode acts as a backup** for metadata.

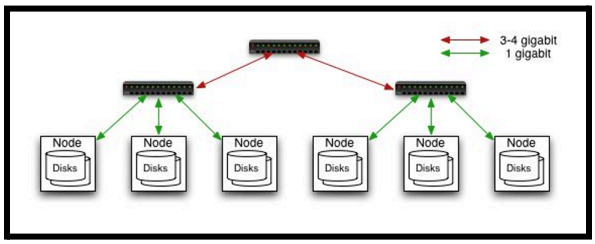
Example:

- If a **DataNode storing important blocks crashes**, the NameNode **detects the failure** and **creates new copies from existing replicas**.

Bottom Line: HDFS automatically detects failures and recovers data without human intervention.

Commodity Hardware Cluster in HDFS

- Hadoop **runs on commodity hardware** (low-cost servers instead of expensive enterprise storage).
- Typical cluster structure: Typically in 2 level architecture
  ✅ **40 nodes per rack**.
  ✅ **3-4 gigabit uplink per rack**.
  ✅ **1 gigabit internal network speed**.



Why is this beneficial?

✅ **Low Cost** → No need for expensive high-end servers.
✅ **Easier to Scale** → More machines can be added as needed.
✅ **Reduces Downtime** → If a server fails, another takes over.

MAP REDUCE

What is MapReduce?

Definition:

- **MapReduce is a programming model used to process large datasets in parallel across a Hadoop cluster.**
- **HDFS stores the data**, but **MapReduce processes the data** to extract meaningful insights.
- **MapReduce is not a programming language**, but rather a **framework for writing data processing logic**.

Why is MapReduce Needed?

✅ **Big Data is too large to process on a single machine** → We need a **parallel computing approach**.
✅ **Traditional data processing is slow** → MapReduce **divides tasks into smaller chunks** and processes them in parallel.
✅ **Scalability** → Works efficiently on **hundreds or thousands of nodes**.

Bottom Line: MapReduce allows us to process massive datasets efficiently using distributed computing

How Does MapReduce Work?

MapReduce consists of two main functions:

1️⃣ **Map Function** → Processes and transforms raw data into key-value pairs.
2️⃣ **Reduce Function** → Aggregates and summarizes the processed data.

🔹 **Hidden Step: Shuffle and Sort** → Organizes data between **Map and Reduce stages**.

Map Function: Breaking Down the Data

What Does the Map Function Do?

✅ Runs **first** in every MapReduce job.
✅ **Extracts useful information** from the raw dataset.
✅ Breaks down large input into **smaller, manageable pieces**.
✅ Outputs **key-value pairs** → These become the **input** for the Reduce function.

📌 **Example: Processing Stock Data**

- Suppose we have **stock market transaction data** in the following format:

plaintext Copy Edit

```
TICKER, PRICE, TIMESTAMP
AAPL, 150.25, 10:05
MSFT, 299.50, 10:06
AAPL, 152.00, 10:07
```

- **Map Function extracts the stock symbol and price:**

plaintext Copy Edit

```
(AAPL, 150.25)
(MSFT, 299.50)
(AAPL, 152.00)
```

Bottom Line: The Map function processes raw data and extracts meaningful key-value pairs.

Reduce Function: Aggregating the Data

What Does the Reduce Function Do?

✅ **Aggregates and summarizes data** produced by the Map function.
✅ Performs operations like **counting, summing, averaging, or finding the maximum/minimum values**.
✅ Not always required → Some jobs **only need the Map step (Map-Only Jobs)**.

📌 **Example: Computing Average Stock Price Per Hour**

- The **Reduce function** will take the key-value pairs from the **Map function** and compute averages.
- **Input to Reduce Function:**

plaintext Copy Edit

```
(AAPL, [150.25, 152.00, 153.50])
(MSFT, [299.50, 301.00, 298.75])
```

- **Reduce Function Computes Average:**

plaintext Copy Edit

```
(AAPL, 151.91)
(MSFT, 299.75)
```

Bottom Line: The Reduce function aggregates data into meaningful insights.

Hidden Step: Shuffle and Sort

Why is Shuffle and Sort Needed?

- **Between the Map and Reduce steps, the data must be organized properly.**
- The **Shuffle and Sort phase** ensures:
  ✅ **All values related to the same key are grouped together.**
  ✅ **Data is sorted** before being sent to the Reduce function.

📌 **Example: How Shuffle and Sort Works**

- If the Map function produces:

plaintext Copy Edit

```
(AAPL, 150.25)
(MSFT, 299.50)
(AAPL, 152.00)
(AAPL, 153.50)
(MSFT, 301.00)
```

- **Shuffle and Sort Organizes Data:**

plaintext Copy Edit

```
(AAPL, [150.25, 152.00, 153.50])
(MSFT, [299.50, 301.00])
```

Bottom Line: Shuffle and Sort organizes data between the Map and Reduce steps to ensure efficient processing.

<u>MapReduce follows the same logic as Unix Pipes:</u>

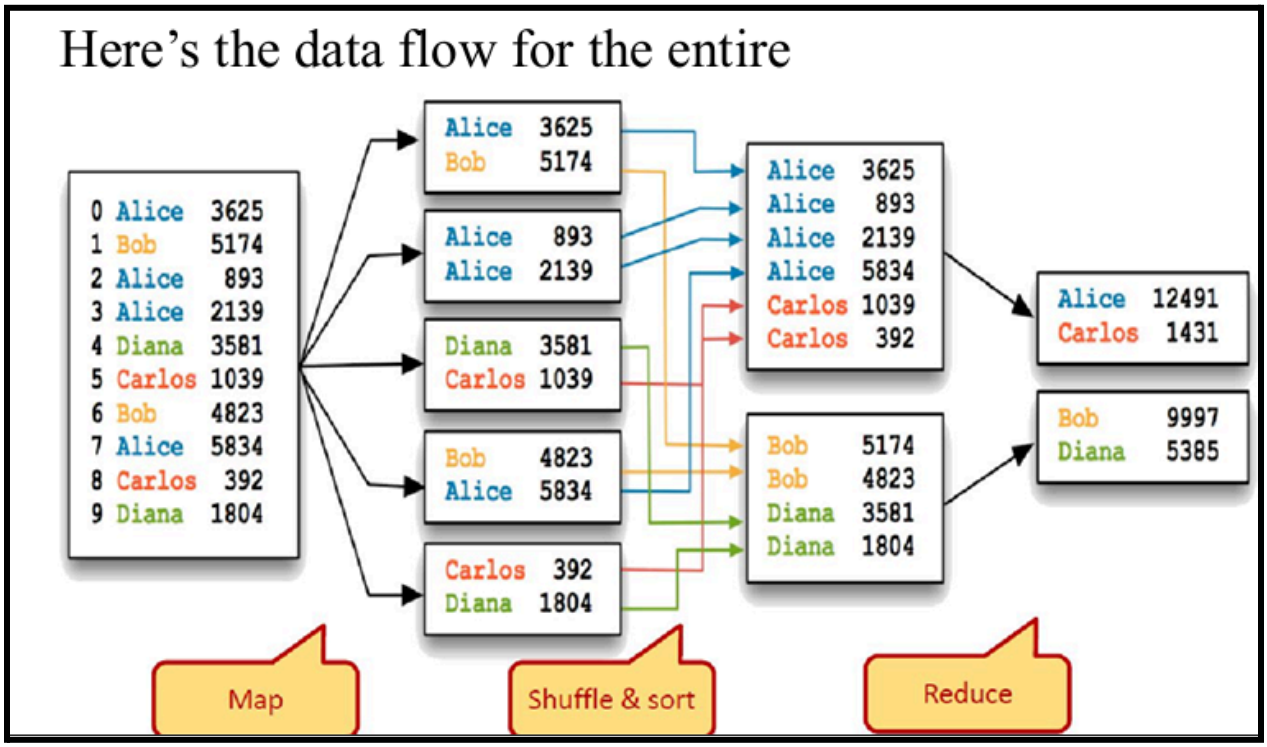
- Unix Pipes take the output of one command as input to another command.
- Similarly, Map outputs become Reduce inputs.

Bottom Line: MapReduce is inspired by Unix pipes, where data flows through multiple stages before producing a final result.

When to Use MapReduce?

<u>Best suited for:</u>

✅ Processing massive datasets that do not fit on a single machine.
✅ Log analysis, clickstream data, social media analytics, fraud detection.
✅ Counting and aggregation tasks (word count, stock price analysis, website traffic).
✅ Sorting and filtering large data files.

<u>Not ideal for:</u>

❌ Real-time processing (MapReduce jobs take time to execute).
❌ Small datasets (Traditional SQL queries are faster).
❌ Iterative algorithms (Machine Learning prefers Spark over MapReduce).